Daniel Borkovec: 2941124B
Haider Alboloushi: 2961047A

Digital Signal Processing – Finite Impulse Filters Investigation

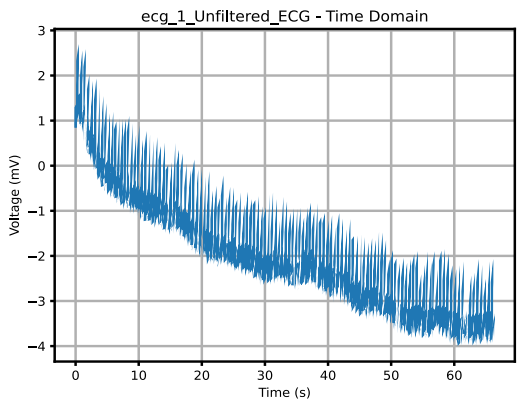
Introduction and Objectives

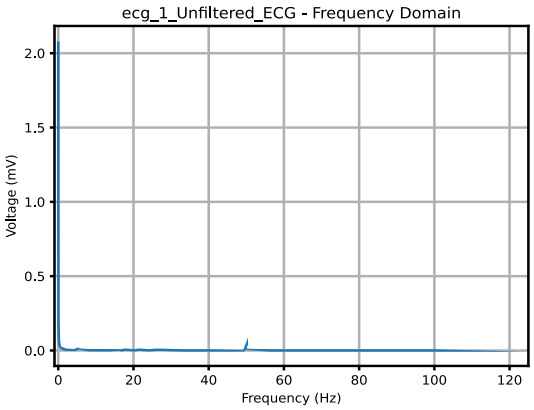
This report covers the second assignment of the DSP course; digital, semi-real time filtering of an Electrocardiogram signal using Finite Impulse Response filtering, referred to as ECG and FIR, respectively. Unlike the previous audio-enhancement process in Assignment 1 which utilized a non-causal process, the Discrete Fourier Transform, the FIR process is causal, meaning analyzing the whole input data at once is not a requirement. Essentially, the FIR filter may continuously be fed input data sample-by-sample and, following a slight delay which will be discussed later, outputs the filtered data in semi-real time. This makes the FIR filter a perfect method for filtering continuously sampled data, such as biomedical signals, live broadcasts, or even seismic monitoring.

This lab follows the step-by-step development and implementation of various FIR filters for simulating real-time ECG filtering using pre-recorded data. The intended objectives and outcomes of this lab are the following: demonstrate plotting in the time and frequency domain, write a function to calculate FIR coefficients efficiently, implement FIR filtering in a Python Class wrapper for modularity, compare frequency response FIR filtering and LMS filtering and use matched filters to identify the exact timing of R peaks to infer the momentary BPM reading.

Part 1 - Plotting and Initial Analysis

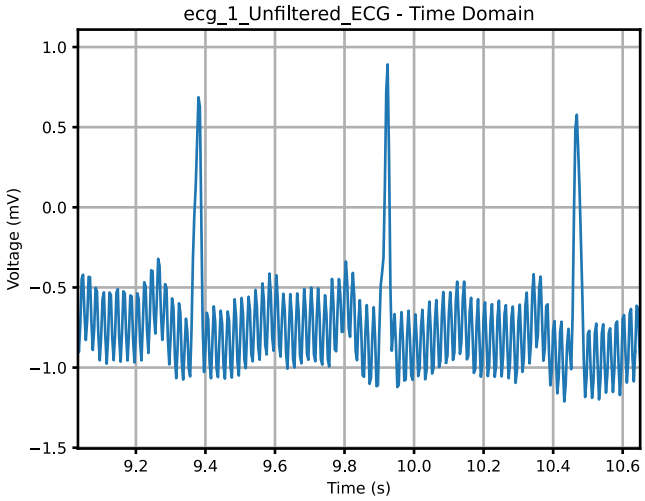
Firstly, both lab partners, Daniel and Haider, had their ECG recorded in the lab to generate realistic, and relevant, ECG data. Although the data will be pre-recorded, and thus not real-time, a real-time scenario may be simulated later by passing the data sample-by-sample to the FIR filter. Subsequently, the NumPy and Matplotlib libraries were used to parse the .tsv files and accurately plot the data in the time and frequency domain. Both the time and frequency visualizations of Haider's ECG may be seen below on the left and right, respectively:





Upon immediate inspection, the time domain data shows a clear DC wander, as the peaks go from a 3 mV maximum at the start down to -2 mV peaks at the end. This is a direct consequence of the human skin's many fluctuating properties impacting its electrical potential, including sweat, muscle signals, and includes the shifting contact of the electrode with the skin. This DC wander is clearly represented in the frequency domain as an over 2 mV peak in the sub 1 Hz frequency bins. Keeping the y-axis in a linear scale shows the vast difference between the amplitude of the signal containing the DC wander and the actual heartbeat data, located roughly between 0.5 Hz and 40 Hz or higher depending on the intended quality, indicating how critical it is for the DC wander to be filtered to analyze the PQRST data clearly. Thus, the first frequency range to be removed lies the sub ~1 Hz region.

Additionally, the time domain signal shows a "thick band" of interference located on the bottom half of the waveform. This region predominantly hosts the periodic, constant in amplitude 50 Hz mains interference contaminating the signal. The interference is also indicated on the frequency domain plot as the ~0.2 mV peak at exactly 50 Hz, which follows the UK's 50 Hz mains frequency. In other parts of the world, this contamination would lie at different frequencies, for example, 60 Hz for the US, meaning a truly universal heartbeat monitor's mains rejection band needs to be modular in nature. For this lab, 50 Hz rejection is sufficient. Building on this, the mains noise is clearly visible when zooming into the time-domain signal, as almost all PQ and ST parts of the heartbeat are drowned out in the 50 Hz noise, as illustrated on the plot below:



As a consequence of the 50 Hz signal, its harmonics at N * 50 Hz, may also contaminate the signal. However, attenuating higher frequencies should be done with care, as the sharp transitions of the heartbeat, such as the R peaks, are also comprised of higher frequencies, and thus their attenuation would risk distortion to the signal. For us, adding 95 Hz – 105 Hz attenuation to the filter, that is attenuating the second harmonic of 50 Hz band, did not result in any meaningful noise attenuation. Moreover, any further harmonics above 100 Hz are above the Nyquist frequency of 250 / 2 = 125 Hz and thus have aliased into the signal and may not be removed digitally. As such, we chose to only attenuate the mains noise's fundamental, 45 – 55 Hz for the UK, which made the biggest impact in noise reduction and purposefully chose to ignore any further harmonics to avoid risking loss of detail in the heartbeat complex itself.

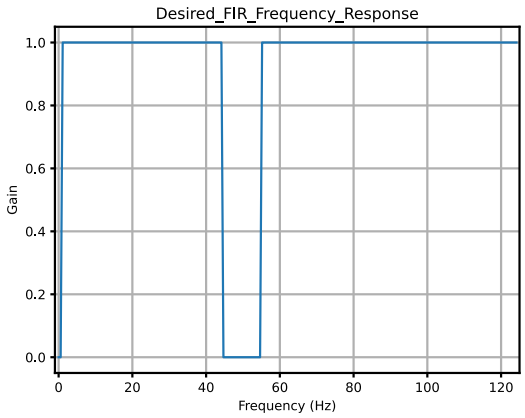
Part 2 – FIR Design

Beginning the filter design, the FIR filtering process requires a set of two arrays; a buffer of length **M,** which is continuously updated with new input data, and an identically sized **M-**length array of "FIR coefficients". The necessity for the two arrays stems from the mathematical implementation of the filter, which requires multiplying each input-coefficient pair and summing the results to yield the filter output. Therefore, a method of choosing the appropriate **M**, called the "*tap number*", and the corresponding **M** number of coefficients is crucial for effective FIR operation. Borrowing from the lectures, the methodology of **M** calculation was done using the kaiser method. Although the kaiser method is empirical, it has been battle tested to work effectively in real-life scenarios.

The kaiser method uses two definable parameters, the intended side lobe decay and the transition width, and calculates the required **M** accordingly. For medical purposes, a doctor examining a patient's ECG would require it to be not only close to real-time, but adequately noise-free. Accordingly, we chose a side lobe decay of 60 dB and an associated transition width of $\frac{2\pi*2}{Fs} = 50ms$ as a compromise between noise suppression, signal preservation and group delay. Using the two above-specified parameters, the following **M** number and FIR properties are achieved:

- The Kaiser-calculated tap number, **M**, is: **453**
- resulting in a frequency resolution of: **0.552 Hz**
- and group delay of: **0.906 seconds**

The FIR properties thus achieve not only semi-real response time, a delay of 0.906 seconds, but also a reasonable resolution of 0.552 Hz for dealing with heartbeat signals. In any case, our modular **M** calculation allows for easy modifications should the filter requirements change. Subsequently, the filter's impulse response to remove the undesired frequencies needs to be generated to fill one of the **M**-sized arrays with corresponding FIR coefficients. First, the filter's desired frequency response is manually constructed in the frequency domain. Secondly, using an inverse FFT, it is transformed into the time domain. The following plot below illustrates the desired frequency response, or the Bode Plot, of the filter to remove the DC wander, 0.1 Hz – 0.6 Hz, as well as the 50 Hz interference, 45 Hz – 55 Hz (mirror frequencies removed for plotting):



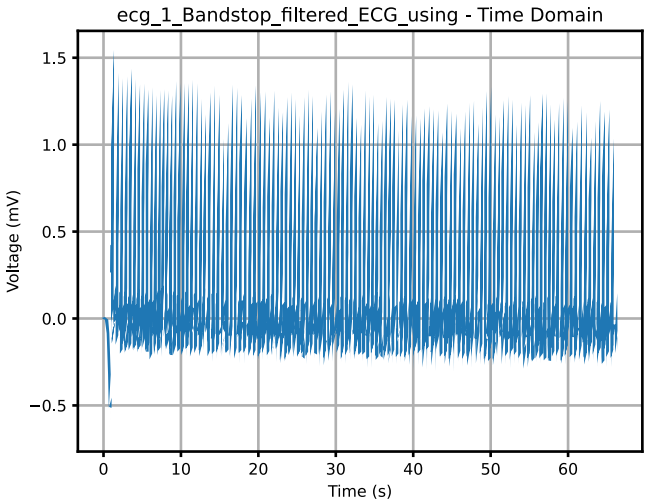
The DC wander frequencies were chosen carefully to not attenuate the low-frequency T components of the heartbeat waveform. Importantly, the generated frequency response is perfectly symmetrical along the **M** / 2 middle point, which means impulse response, is also symmetrical. This is crucial in preserving the waveform through the filtering process, as symmetric FIR coefficients result in a linear-phase filter, which delays all frequencies the same.

Thirdly, to transform the raw IFFT time-domain data, **x**, into the correct impulse response, **h**, the first half of **x** is moved into the second half of **h** and vice versa, preserving the correct symmetry. Lastly, a kaiser window function is applied to the final impulse response, **h**, to improve the spectral leakage resulting from the filtering operation. For ease of use, the whole process is then wrapped into a single function called: ***calculate_FIR_coefficients(****lobe_decay, transition_width****)***

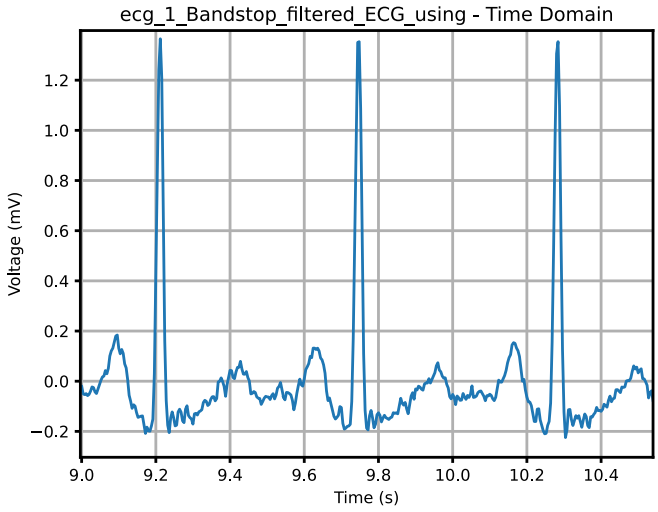
Part 3 – FIR Implementation

Continuing, a comprehensive "***FIRfilter***" class was constructed, with internal methods implementing the actual sample-by-sample filtering process. For initialization, an instance of the class requires an array of pre-calculated FIR coefficients, which will be passed to it from the previously defined ***calculate_FIR_coefficients()*** function. The ***FIRfilter*** class then holds a "***doFilter***" method, which takes the next sample of the input, pushes it into its **M** length buffer. Then, the function multiplies each input data in the buffer with the corresponding FIR coefficient, and returns the full sum as an output, thus implementing FIR filtering. Internally, this method uses an instance of our custom "***CircularBuffer***" class for its input buffer, which as the name suggests utilizes a circular buffer instead of a shift register buffer. This buffer implementation heavily speeds up the computational overhead every time a new sample is added to the buffer.

After an ***FIRfilter*** instance is initialized using the appropriate coefficients, the ***doFilter*** method is called for every ECG sample individually inside a range based *for* loop, simulating sample-by-sample filtering of a continuous, real-time signal. The following graph illustrates Haider's resulting time-domain ECG after all samples were filtered and compiled into an output array:



Upon immediate inspection, the DC wander previously observed is no longer present and the 50 Hz interference, previously creating a much stronger band on the lower part of the waveform, is substantially attenuated. Furthermore, the beginning illustrates how the FIR filter's output response is not fully effective until the buffer fills at, in our case, $\frac{M}{Fs} = \frac{453}{250\ Hz} = 1.81\ seconds$. After it is fully filled, the FIR filter applies the impulse response from its **M** taps in full capacity. Zooming into the three PQRST peaks centered around 9.8 seconds yields the following figure:



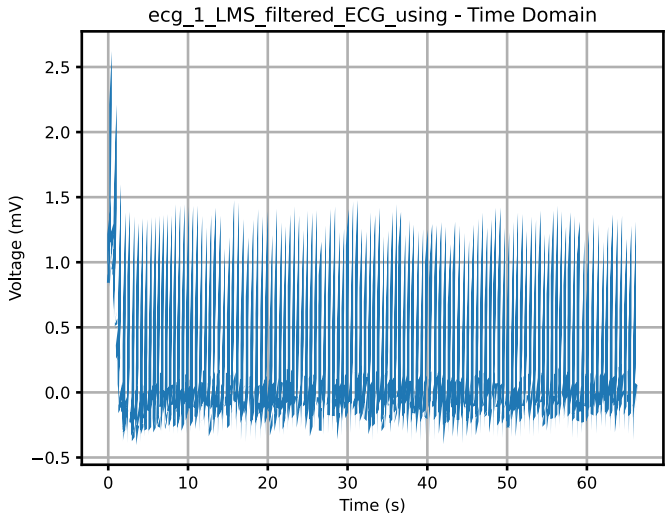
Compared to the previous unfiltered PQRST waveform visualized in **Part 1**, the PQRST waveform is clearly distinguishable. Although slight higher-frequency noise is still present, the zoomed figure clearly illustrates the significant attenuation of the previously overwhelming 50 Hz interference, as the PQ and ST parts are now visible. Most importantly, the information which a doctor would be looking for in the ECG, the end of the R and beginning of the T parts, are distinguishable and thus this filter may theoretically be used for basic medical evaluation by a trained professional.

Part 4 – "Least Mean Squared" Implementation

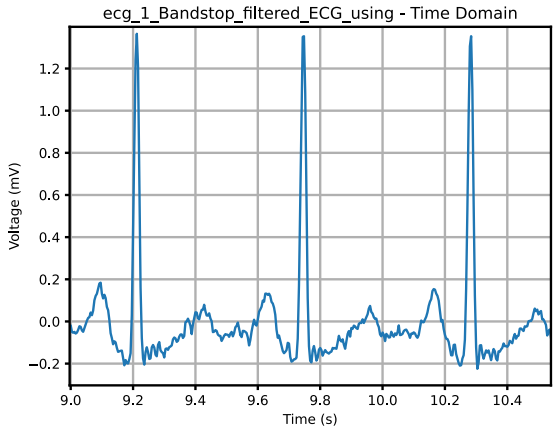
Having examined basic impulse response FIR filtering, the ***FIRFilter*** class was expanded with one more method, called ***"doFilterAdaptive".*** As the name suggests, this version of the method also filters an input signal, however, the filter slowly alters, or *adapts*, its FIR coefficients to iteratively rid the input of a specific sample noise instead of requiring pre-calculated coefficients.

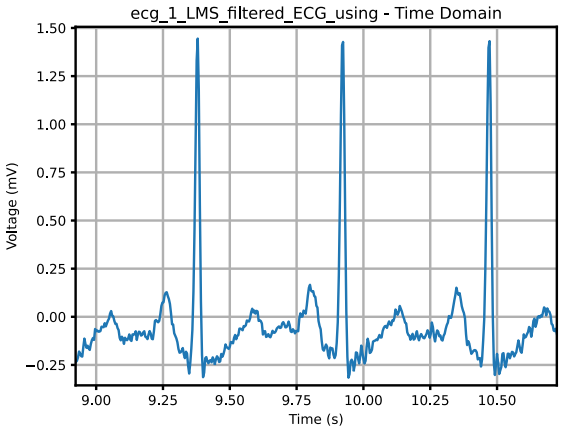
In contrast to the previous approach, Least Mean Squared filter requires two inputs for filtering, a noise input and a signal input. Internally, the method first calls the ***doFilter*** method using the noise as input, thereby filling the input buffer with noise data rather than signal data, in direct contradiction to the previous method. Subsequently, it generates an "***error***" variable using the difference between the output of ***doFilter*** and the second input, the signal. ***Error*** also also serves as the filter output. Lastly, using a ***learningRate*** variable, it updates all FIR coefficients to minimize the ***error***, thereby improving the noise removal in the next ***doFilterAdaptive*** iteration.

With the goal of removing DC wander and 50 Hz interference using this method, a sine wave at 50 Hz was generated at each **M** sample and then offset by a constant value, serving as the noise input simulating mains interference and fluctuating skin potential. After some testing, we found a +0.2 offset was the ideal balance between removing DC wander while retaining detail in the horizontal parts of the PQRST complex. Thus, the artificial noise now includes the two undesired frequency components, DC and 50 Hz, which we wish to remove and will henceforth be passed alongside the input to the ***doFIlterAdaptive*** at every sample. The figure below illustrates the time-domain plot of ecg_1, Haider's ECG, filtered using the LMS filter method:



Similarly to the frequency response method, which required the **M** buffer to fill for ideal filtering quality, the LMS filter requires multiple iterations to adapt its FIR coefficients adequately. This phenomenon is visible as the large DC wander still being present in the beginning, reaching ~2.8 mV, due to the FIR coefficients not yet being adapted. Specifically, we found that the LMS output stabilizes at approximately 2 seconds, which is directly comparable with the 1.81 seconds required to fill the input buffer. Nevertheless, once the filter had had enough samples to stabilize, we found the waveform to be slightly sharper than the frequency response method. The zoomed figure below illustrates the same three PQRST peaks centered at 9.8 seconds for easier comparison, with the previous method on the left and the LMS method on the right:

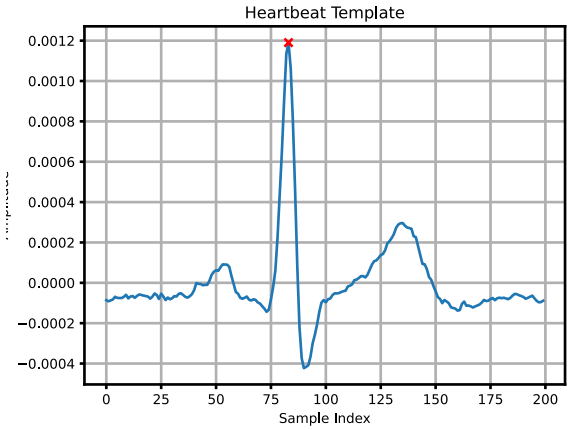




As before, the full PQRST complex is clearly visible. However, when compared to an actual PQRST complex, the LMS filter manages to retain more subtle details. For example, contrarily the frequency response method which cuts off the slight Q and S negative dips, the LMS filtered waveform retains them alongside the, almost, horizontal PR and ST sections. Additionally, the smaller Q and T peaks contain visible less high-frequency jitter than the previous method. In consequence, given both methods took as similar amount of time to implement, we found the LMS method to generate a slightly sharper and more detailed waveform and thus consider it superior in our simple laboratory demonstration.

Part 5 – Heartbeat Detection Using Matched Filters

In the last section of this lab, a matched FIR filter method was used to detect the R peaks in the input ECG and thus calculate the momentary heart rate using their time differences. In yet another shift in implementation, the matched filter works based on a ***template*** which is continuously compared with the input and produces a large spike at the output when a matching section is detected. Therefore, when using a PQRST complex as a template, it follows that the filter produces a large spike exactly when another PQRST complex is detected, serving as our metric for detecting individual heart beats. In the case of this lab, we used Daniel's filtered ECG to isolate a single PQRST to use as the ***template***, which is visualized on the figure below:



Using a multimeter, we found that Daniel's effective resistance between the two electrodes on his ankle and forearm was about 200 kΩ, compared to Haider's 1.1 MΩ. As visible on the figure above, Daniel's ECG is significantly cleaner and lacks much of the higher frequency noise of Haider's ECG analyzed in the sections above, likely as a direct consequence of the lower total resistance, hence the decision to use Daniel's as the **template**.

Following the mathematical definition of this method, the ***template*** was then time-reversed and an ***FIRfilter*** instance was created using it as its FIR coefficients. Then, every matched filter output was squared to improve SNR and passed to our causal ***detect_R_peaks()*** function.

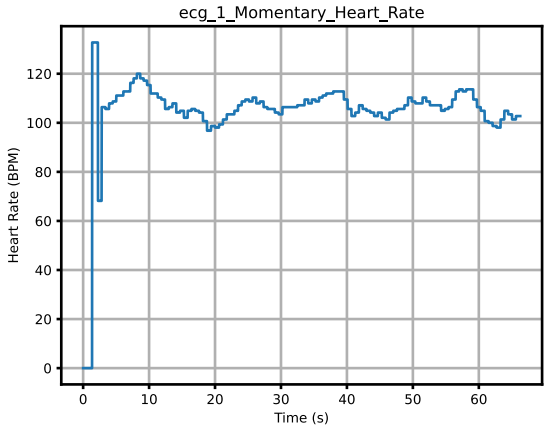
The logic behind this function uses a C-like “static” variable, in Python defined outside the loop, which persists after each loop ends, keeping track of the last ***maximum*** value of the signal. This value decays by a percentage with each sample, which allows subsequent signal maxima to be detected. Once another ***maximum*** was found in a sample, the function waits until the input starts decreasing again, thus locating the *falling edge* of an R peak. Once detected, this triggers the function to output a 1 instead of a 0, thereby creating an array of detections.

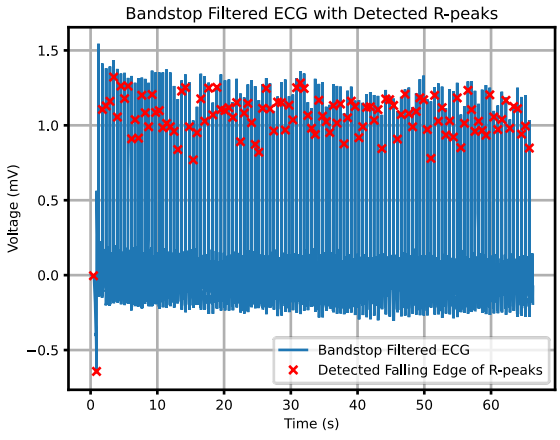
Upon the detection of a peak, then we can count the number of samples between it and the last detection stored as a ‘1’ in the ***detected_R_peaks*** array. Using that difference, it’s simply a matter of converting from a difference in samples to a difference in time using the sampling frequency, in our case 250 Hz, and converting a time difference to BPM, finally yielding a momentary heart rate which updates with every detected peak, correct or otherwise.

To minimize false detections, a heuristic is applied as a last check to discard any obviously incorrect detections. Specifically, a reading is discarded if the resulting resting BPM is larger than 150 BPM, a value we assumed would not occur during resting, which maps to a difference between samples of $\frac{60}{\boldsymbol{Maximum_BPM}} * Fs$. For a more versatile approach, the heuristic limit may be raised to cover all possible healthy heart rates; such as common limit of 220 BPM at maximum.

Crucially, we only discard BPM values exceeding a certain BPM, in other words below a certain sample difference, and do not apply a lower BPM heuristic. The reason is, if the matching filter missed a single heartbeat, every subsequent correct reading would result in an abnormally small BPM and would be discarded. If an external stimulus did not reset the sample counter, then the heuristic would ignore every following BPM reading due to it being too low. Therefore, for our purposes in this experiment, we applied a heuristic only for discarding implausibly fast BPMs.

Connecting it all together, after Hadier’s full recording, ecg_1, was passed through an initial frequency response filter to remove noise, then a matched filter and then the heuristic filter, two outputs were generated. The two figures below indicate both the momentary heartbeat plot, left, as well as the time-domain plot with the detected peaks overlayed on top, right:





As expected, the initial incorrect readings before the input buffer of the preceding filtering layer is filled leads to incorrect heartbeat readings. However, we observed that if the matched filter R peak detection was applied to an unfiltered ECG, the DC wander and interference caused a worrying amount of either missed detections or false positives. Therefore, we decided that a preceding filter was a necessary evil for the detection to work consistently, even though the first few readings would have to be discarded and an additional delay was added.

Nevertheless, once the buffer saturates in 1.81 seconds for our 453 tap number, the detections have a staggeringly high accuracy, indicating that the causal process is effective and, in this case, may be used to correctly generate ECG beeps. It should be noted that both the template detection and filter introduce a slight delay, meaning their use in tandem needs to be calibrated to achieve an adequate level of noise reduction while retaining an acceptable delay.

In essence, the full matched filter chain includes an initial FIR filter group delay 0.906 seconds, as well as a matched filter delay of 0.438 second due to the R peak occurring at the 82nd sample of the ***template's*** 200 samples, resulting in a total matched filter delay of ~1.37 seconds.

As a further consequence of our falling edge detection process, the R peak detections are slightly delayed compared to the true R peak maximum. At minimum, at 250 Hz this delay is only $\frac{1}{250\ Hz} = 4\ ms$, or slightly more if the next R peak sample lands on the falling edge but at a higher amplitude than the previous rising edge sample.

Overall, we concluded that our FIR filter, Matched filter and R peak causal detection chain process successfully extracted the momentary heartbeat even in Haider's noisier ECG, thus proving the method's efficacy in real-life scenarios.

Conclusion

In conclusion, this lab successfully demonstrated the methodology behind various FIR filter variations and their implementations when filtering biomedical ECG data.

First, the kaiser method was used to calculate the number of required FIR coefficients for both DC wander and 50 Hz noise removal, making sure the resulting frequency resolution and group delay was adequate for use in real-life biomedical operations. Then, using a prefect frequency response as a framework, the FIR coefficients were calculated and used to initialize a frequency response FIR filter. When employed to filter Haider's ECG data, this method illustrated both its simplicity in implementation and its effectiveness in removing unwanted interference. However, the investigation revealed this method's inherent requirement to fully saturate the **M**-sized buffer of input data to filter at full efficiency.

Subsequently, a Least Mean Squared filter approach was investigated, with an artificial 50 Hz offset sine wave being used to train the FIR coefficients. Although the same inaccuracy in the output towards the beginning of the samples was observed, comparing the overall results to the previous method illustrated the LMS method's superiority in retaining more of the PQRST complex waveform and thus leading to a more detailed output.

Lastly, a matched filter approach was combined with the frequency response filter to detect the R peaks in the PQRST complex. Isolating a single PQRST complex in Daniel's less noisy data, a template was created and used to generate a binary "match" output of R peak detections. Combining this output with a simple BPM heuristic maximum yielded an overwhelming accuracy in heartbeat detections, illustrating this method's efficacy for detecting predictable patterns in continuous input data.

Appendix – Python Implementation code:

Appendix 1 – ecg_plot.py

```
# Required imports

import numpy as np
import matplotlib.pyplot as plt
import os

# ==============
# Global Defines
# ==============

path_to_files = ''
path_to_output = 'Output_Files/'

# For us the ECG was set to 250 Hz
# For calculations below, the Fs is calculated from the delta Time anyway
sampling_frequency_Hz = 250

# ============================
# === Essential Functions ====
# ============================

def Load_ECG_Data(name_of_file):
    """
    Use NumPy to read the ECG .tsv files and output an array
    """

    # Use the global path to files for an organized directory
    global path_to_files

    # first load the data as a text
    # ignore comments and data separators
    data = np.loadtxt(path_to_files + name_of_file, delimiter="\t", comments="#")

    # Extract time and ECG (time is first column, Channel 2 data is 8th)
    # Both are one-dimensional arrays
    time = data[:, 0]
    ecg = data[:, 8]

    # Convert to miliVolts
```

```
    ecg *= 1000

    sampling_frequency_Hz = 1 / (time[1] - time[0]) # Use Fs = 1 / deltaT

    return time, ecg, sampling_frequency_Hz

def fft(data, samplerate):
    """
    Converts a time domain signal into its frequency domain representation
    """

    # ONLY 1 OF 2 NUMPY HIGH-LEVEL FUNCTIONS
    fft_data = np.fft.fft(data)
    N = len(data)

    # Make empty freq. array of size N
    frequency_axis = np.empty(N)

    # Frequency resolution = Fs / N
    # Frequency = k * resolution

    # fill array with correct frequencies
    for i in range(N):
        frequency_axis[i] = i * (samplerate / N)

    return fft_data, frequency_axis

def inverse_fft(fft_data):
    """
    Converts an FFT back to the time domain using IFFT
    """
    # SECOND OF THE TWO NUMPY HIGH-LEVEL FUNCTIONS
    # Convert back to time and take only real values, as real sound isn't imaginary :) unlike
my best friend :(
    ifft_data = np.real(np.fft.ifft(fft_data))

    return ifft_data

# ============================
# === Plotting Functions ====
# ============================

def plot_time_domain(data_array, time_axis, name_of_plot, show_plot=True, y_label='Voltage
(mV)'):
    """
```

```
    Plots the time domain of a signal using its array
    """

    # Plotting time domain - assume that the ECG data already contains a time axis
    plt.plot(time_axis, data_array)
    plt.title(f'{name_of_plot}')
    plt.xlabel('Time (s)')
    plt.ylabel(y_label)
    plt.grid()
    plt.savefig(path_to_output + name_of_plot + '.svg') # Save as an SVG
    if show_plot:
        plt.show()
    plt.close()

def plot_frequency_domain(fft, frequency_axis, samplerate, name_of_plot, show_plot=True,
normalize=True, y_label='Voltage (mV)'):
    """
    Plots the frequency domain of a signal using its array
    """

    # We used this normalization in the audio assignment, keep the option for now
    if normalize:
        fft = fft / len(fft)

    # FFT is done in voltage
    fft_voltage = np.abs(fft)

    # Plotting frequency domain
    plt.plot(frequency_axis, fft_voltage)
    plt.ylabel(y_label)
    plt.title(f'{name_of_plot}')
    plt.xlabel('Frequency (Hz)')

    # plt.xscale('log') Optionally could log scale X-axis, but the x range isn't large for ECG
data

    # Cutt off the mirror frequencies (above Fs / 2)
    plt.xlim(-1, samplerate / 2) # If using Log on the x-axis, Log can't start at 0 so use 1

    plt.grid()
    plt.savefig(path_to_output + name_of_plot + '.svg') # Save as an SVG
    if show_plot:
        plt.show()
    plt.close()
```

```
def main():

    # Make sure the proper output path is used
    global path_to_output
    os.makedirs(path_to_output, exist_ok=True)

    # =============== DATA PREPARATION ================

    print("\n===============\n")
    print("Running: ECG_Plot\n")

    file_name = "ecg_1.tsv" # Used for Steps 1 - 4

    plot_name = file_name[:-4] # Remove the .tsv for plot naming

    time, ecg, Fs = Load_ECG_Data(file_name)
    print("File: ", file_name)
    print("Sampling Frequency", Fs)
    print("Duration:", time[-1], "seconds")

    # =============== PART 1 - PLOTTING ================


    # Plot the ECG in time domain
    plot_time_domain(ecg, time, plot_name + "_Unfiltered_ECG - Time Domain")

    # Plot the ECG in the frequency domain
    ecg_fft, frequency_axis = fft(ecg, Fs)
    plot_frequency_domain(ecg_fft, frequency_axis, Fs, plot_name + "_Unfiltered_ECG -
Frequency Domain")

    print("\n===============\n")


if __name__ == "__main__":
    main()
```

Appendix 2 – fir_design.py

```
# Required imports

from ecg_plot import *

# ============ ASSIGNMENT PART TWO ====================

def kaiser_variable_helper(A, deltaOmega):
    """
    Calculates the required variables for the Kaiser Window using the formulas in the lectures
    """

    # Implement the empyrical piecewise operation for Beta
    if(A > 50):
        beta = 0.1102 * (A - 8.7)
    elif(A<=50 and A>=21):
        beta = 0.5842 * (A - 21)**0.4 + 0.07886 * (A - 21)
    else:
        beta = 0

    # Cast to integer for later calculations, and add 1 because int() rounds down, and we want an M larger than the "minimum" recommended
    M = int((A - 8) / (2.285 * deltaOmega)) + 1

    # Ensure the FIR filter has an odd number of taps, so the middle index lays at an integer, and NOT a half
    if M % 2 == 0:
        M += 1

    return M, beta


def kaiser_window_function(h, beta, alpha):
    """
    Ammplies a Kaiser Window Function to the input array "h"
    """

    # Generate sample indices from 0 to N-1
    n = np.arange(len(h))

    # Calculate the Kaiser window from the mathematical formula in the lecture slides
    window = np.i0(beta * np.sqrt(1 - ((n - alpha) / alpha)**2)) / np.i0(beta)
```

```
    # Apply the Kaiser window by multiplying h
    h *= window

    return h

def calculate_FIR_coefficients(bandstop_frequency_array, sampling_rate, remove_DC = True,
side_lobe_decay_dB=60, transition_width_rad=0.05):
    """
    Calculates the FIR coefficients

    Bandstop_frequency_array should be a two-dimensional array of integer pairs, representing
the start and stop frequencies of the stopbands

    Example:
        stopbands = np.array([
            [45, 55],     # Remove 50 Hz interference
            [55, 65],     # Remove 60 Hz interference (if we want to also remove US mains
noise for example)
        ])
    """

    # Calculate M from the Kaiser method.
    # If A and deltaOmega aren't specified in the function arguments, use the defaults we
found most effective
    M, beta = kaiser_variable_helper(side_lobe_decay_dB, transition_width_rad)

    # Also report the resulting delay and resolution
    group_delay = M / (2*sampling_rate)
    print(f"\nThe Kaiser-calculated tap number, M, is: {M} \n\t resulting in a frequency
resolution of: {(sampling_rate / M ):0.3f} Hz \n\t and group delay of: {group_delay}
seconds\n")

    # Calculate the middle index (M is always odd) -> cast to integer to round down to the
correct index
    middle = int(M / 2)

    # prepare the variable to store all stopbands' frequency representation
    final_frequency_window = np.ones(M)

    for stopband in bandstop_frequency_array:

        # calculate the correct position of the stopband in terms of tap number M
        k1 = int( (stopband[0] / sampling_rate) * M)
        k2 = int( (stopband[1] / sampling_rate) * M)
```

```
        # always maintains symmetry around the M/2 point -> linear phase filter to maintain constant group delay!
        final_frequency_window[k1: k2+1] = 0 # zero out stopband in the shared frequency window
        final_frequency_window[M - k2: M - k1+1] = 0 # zero out mirror stopband in the shared frequency window

    if remove_DC:
        final_frequency_window[0] = 0 # zero out first tap -> DC HAS NO MIRROR!

    # Plot the window to check
    frequency_axis = np.arange(middle) * sampling_rate / M # Create frequency axis in Hz (positive frequencies only -> up to "M/2")
    # Don't normalize this, as we want to see the 0 dB for full pass amplitude -> unlike real voltage signals which we normalize for graphing by 1 / N
    plot_frequency_domain(final_frequency_window[0 : middle], frequency_axis, sampling_rate, "Desired_FIR_Frequency_Response", normalize=False, y_label="Gain")

    # Move the window from the frequency domain to the tap domain: X[K] -> x[n] & ensure it is real
    x = inverse_fft(final_frequency_window)

    # prepare the Impulse Response array
    h = np.zeros(M)

    # Into the first half move the second half of the Impulse Response x[n] AND VICE VERSA
    # Use the correct center calculated above as M is always odd
    h[0 : middle] = x[middle + 1: M]
    h[middle : M] = x[0 : middle + 1]

    # Apply the Kaiser window function for the intended filter response
    h = kaiser_window_function(h, beta, middle)

    return h
```

Appendix 3 – fir_filter.py

```
from ecg_plot import *

# A CircularBuffer handler to speed up the coefficient buffer, and not have to shift the whole buffer each calculation
class CircularBuffer:
    def __init__(self, size):
        self.size = size
        self.buffer = np.zeros(size) # Fill the buffer with zeros first
        self.oldest_index = 0
        self.fill_count = 0

    def append(self, item):
        """
        Uses the internal variables to correctly loop through the buffer
        """

        # Assign the new value, "item", into the correct position using the counter modulo'd with the total size
        self.buffer[(self.oldest_index + self.fill_count) % self.size] = item

        # The main looping logic
        if self.fill_count == self.size:
            # If the buffer is full, "count == size", then use the "oldest_index" variable to point to the correct position to be overriden
            self.oldest_index = (self.oldest_index + 1) % self.size  # Overwrite oldest
        else:
            # Else the buffer isn't full yet, so keep increasing "fill_count"
            self.fill_count += 1

    def get_item(self, position):
        """
        Calculates the correct index based on the intended position argument
        To return the correct element
        """

        # Start at the newest sample, then move backwards by the requested item position
        index = (self.oldest_index + self.fill_count - 1 - position) % self.size

        return self.buffer[index]

class FIRfilter:
```

```
    def __init__(self,_coefficients):
        self.tap_number = len(_coefficients) # Initialize M to the size of the coefficient
array
        self.FIR_coefficients = _coefficients # Initialize the coefficients to the ones passed
to the contructor
        self.data_buffer = CircularBuffer(self.tap_number) # prepare the Impulse Response
CircularBuffer class of length M -> it starts empty by default

    # Take in the latest value and output the filter result
    def doFilter(self, latest_data):
        """
        In the bandstop filter implementation,
        The delay buffer stores the signal and uses pre-taught coefficients to remove unwanted
noise
        """

        # Append the latest value to the circular buffer
        # The class internally keeps track of the pointer position and overides the correct
value
        self.data_buffer.append(latest_data)

        # Prepare the sum -> scalar value (the filter output)
        sum = 0

        # Multiply, and sum, the corresponding data and n-delay pairs
        for i in range(self.tap_number):
            sum += self.data_buffer.get_item(i) * self.FIR_coefficients[i]

        return sum

    # Filter while changing the FIR coefficients using the "least mean square" error^2
minimization approach
    def doFilterAdaptive(self, signal, noise, learningRate = 0.001):
        """
        In the LMS filter implementation,
        The delay buffer stores the noise and slowly teaches itself how to remove it from the
signal by updating the FIR coefficients
        """

        # Filter the reference noise to estimate interference
        # Starts at 0 and slowly builds to the actual interefence
        estimated_noise = self.doFilter(noise)

        # Calculate the error (cleaned ECG)
        # Starts equal to original ECG and evolves into original ECG - noise
```

```
        error = signal - estimated_noise

        # Then, update all FIR coefficients
        for i in range (self.tap_number):
            # implement deltaH = mu * error * x(n - M)
            self.FIR_coefficients[i] += learningRate * error * self.data_buffer.get_item(i)

        return error
```

Appendix 4 – fir_realtime_sim.py

```
from fir_filter import *
from fir_design import *

path_to_output = 'Output_Files/'

def main():

    # Make sure the proper output path is used
    global path_to_output
    os.makedirs(path_to_output, exist_ok=True)

    # =============== DATA PREPARATION ================

    print("\n===============\n")
    print("Running: Realtime FIR Simulation\n")

    file_name = "ecg_1.tsv"

    plot_name = file_name[:-4] # Remove the .tsv for plot naming

    time, ecg, Fs = Load_ECG_Data(file_name)
    print("File: ", file_name)
    print("Sampling Frequency", Fs)
    print("Duration:", time[-1], "seconds")

    # =============== FIR COEFFICIENT CREATION ================


    # Prepare the stopbands -> remove DC and remove mains noise
    stopbands = np.array([
                [45, 55],     # Remove 50 Hz interference
                [0.1, 0.6]    # Remove DC wander and other factors like breathing (works
really well!) -> careful not to remove the T-part of the PQRST
            ])

    # For the Kaiser Variables, use reasonable values
    FIR_coefficients = calculate_FIR_coefficients(stopbands, Fs, side_lobe_decay_dB=60,
transition_width_rad= 2 * np.pi * 2 / Fs)

    # =============== FIR CLASS USAGE ================

    # Create a bandstop filter instance and empty output array
    Bandstop_Filter = FIRfilter(FIR_coefficients)
```

```
    Bandstop_filter_output = np.zeros(len(ecg))

    # Simulate FIR filtering by passing datapoints one by one and store them in filter_output
    for i in range(len(ecg)):
        Bandstop_filter_output[i] = Bandstop_Filter.doFilter(ecg[i])

    plot_time_domain(Bandstop_filter_output, time, plot_name + "_Bandstop_filtered_ECG_using - Time Domain")

    print("\n===============\n")


if __name__ == "__main__":
    main()
```

Appendix 5 – lms_filter.py

```
from fir_filter import *

path_to_output = 'Output_Files/'

def main():

    # Make sure the proper output path is used
    global path_to_output
    os.makedirs(path_to_output, exist_ok=True)

    # =============== DATA PREPARATION ================

    print("\n===============\n")
    print("Running: Realtime LMS FIR Filter Simulation\n")

    file_name = "ecg_1.tsv"

    plot_name = file_name[:-4] # Remove the .tsv for plot naming

    time, ecg, Fs = Load_ECG_Data(file_name)
    print("File: ", file_name)
    print("Sampling Frequency", Fs)
    print("Duration:", time[-1], "seconds")

    # =============== Least Mean Squared Error Filter ================

    # Create an LMS filter instance and empty output array
    LMS_Filter = FIRfilter(np.zeros(500))
    lms_filter_output = np.zeros(len(ecg))

    # Simulate Adaptive FIR filtering by passing datapoints one by one and storing them in
lms_filter_output
    for i in range(len(ecg)):
        # Introduce a perfect 50 Hz sine as the training noise
        mains_noise = np.sin(2.0 * np.pi * 50 / Fs * i) + 0.2

        lms_filter_output[i] = LMS_Filter.doFilterAdaptive(ecg[i], mains_noise,
learningRate=0.0005)

    plot_time_domain(lms_filter_output, time, plot_name + "_LMS_filtered_ECG_using - Time
Domain")
```

```
    print("\n===============\n")

if __name__ == "__main__":
    main()
```

Appendix 6 – heartbeat.py

```
from fir_filter import *
from fir_design import *

path_to_output = 'Output_Files/'

def detect_R_peaks(filter_output_sample, min_sample_spacing, was_previous_max,
samples_to_prev_detection, max, Fs):
    """
    Causal R peak detection
    Takes in a sample and then compares with the heuristic and previous nmax to detect peaks
    """

    # Assume no detection by default
    r_detected = 0

    # Slowly decay the max value by 0.5% for each sample
    # Expected level before next heartbeat is about 50% (about 136 samples for 110 BPM at 250
Hz Fs)
    # High enough to miss noise, low enough so smaller Matched Filter detections still get
detected
    # Decays more slowly with a lower value
    max = max - 0.005 * max

    # If there is a new maximum value keep track
    if filter_output_sample > max:
        max = filter_output_sample
        was_previous_max = True

    # If the value keeps increasing, keep updating the max, but if it starts falling, detect
the falling edge
    if filter_output_sample < max and was_previous_max:
        # Apply the heuristic
        if( samples_to_prev_detection >= min_sample_spacing):
            r_detected = 1 # New R detection

        # Detect each falling edge only once
        was_previous_max = False

    return r_detected, max, was_previous_max


def main():
```

```
    # Make sure the proper output path is used
    global path_to_output
    os.makedirs(path_to_output, exist_ok=True)

    # =============== DATA PREPARATION ================

    print("\n===============\n")
    print("Running: Realtime Bandstop FIR Filter + Heart Beat Detection Simulation\n")

    file_name = "ecg_1.tsv" # Used for Steps 1 - 4

    plot_name = file_name[:-4] # Remove the .tsv for plot naming
    time, ecg, Fs = Load_ECG_Data(file_name)
    print("File: ", file_name)
    print("Sampling Frequency", Fs)
    print("Duration:", time[-1], "seconds")

    template_file_name = "ecg_2.tsv" # Template used for Matching in Step 5
    template_name = template_file_name[:-4] # Remove the .tsv for plot naming
    template_time, template_ecg, template_Fs = Load_ECG_Data(template_file_name)

    # =============== FIR COEFFICIENT CREATION ================

    # Prepare the stopbands -> remove DC and remove mains noise
    stopbands = np.array([
                [45, 55],     # Remove 50 Hz interference
                [0.1, 0.6]      # Remove DC wander and other factors like breathing (works
really well!)
            ])

    # For the Kaiser Variables, use reasonable values
    FIR_coefficients = calculate_FIR_coefficients(stopbands, Fs, side_lobe_decay_dB=60,
transition_width_rad= 2 * np.pi * 2 / Fs)

    # =============== Matched Filter ================

    # ===== Step 1 - filter the signal(use other lab partner) =====

    print("Template File: ", template_file_name)

    # Use the same coefficients as PART 3 to remove mains and DC wander
    Template_Filter = FIRfilter(FIR_coefficients)
    Template_Filter_Output = np.zeros(len(template_ecg))
```

```
    # Simulate FIR filtering by passing datapoints one by one and store them in filter_output
    for i in range(len(template_ecg)):
        Template_Filter_Output[i] = Template_Filter.doFilter(template_ecg[i])

    # ===== Step 2 = create template from filtered data =====

    # Use daniel_1: 2.9s - 3.7s, convert to samples using sample_rate
    template = Template_Filter_Output[int(2.9 * template_Fs) : int(3.7 * template_Fs)]

    # ===== Step 3, time reverse the template to get the Impulse Response =====
    Matched_FIR_coefficients = template[::-1]

    # ===== Step 4, filter the ORIGINAL ECG with the time-reversed template =====

    # Prepare the Matched Filter instance as well as a bandstop filter instance to feed it
filtered data
    Matched_FIR_Filter = FIRfilter(Matched_FIR_coefficients)
    Input_Bandstop_Filter = FIRfilter(FIR_coefficients)

    # Prepare all output arrays
    Matched_FIR_Filter_Output = np.zeros(len(ecg))
    Input_Bandstop_Filter_Output = np.zeros(len(ecg))
    detected_R_peaks = np.zeros(len(ecg))
    momentary_heart_rate = np.zeros(len(ecg))

    # Create static variables
    local_maximum_nmax = 0
    samples_to_prev_detection = 0
    was_previous_max = False
    R_detected_once = False

    # Heuristic array -> only accept detections matching valid bpm
    # As the R_peak detection is causal, we can't really ignore peaks which fall into a
smaller bpm
    # Once the bpm would be lower than 40 bpm, then no more values would ever be taken
    # Therefore, APPLY ONLY MAXIMUM BPM HERE
    Maximum_Heuristic_BPM = 150

    # Convert BPM thresholds to an accepted min spacing:
    min_sample_spacing = 1 / (Maximum_Heuristic_BPM / 60) * Fs

    for i in range(len(ecg)):

        # Keep track of previous detections
        samples_to_prev_detection += 1
```

```
        # First filter the original ECG using a normal bandpass
        Input_Bandstop_Filter_Output[i] = Input_Bandstop_Filter.doFilter(ecg[i])

        # Use the filtered ECG output as the input to the Matched filter
        Matched_FIR_Filter_Output[i] =
Matched_FIR_Filter.doFilter(Input_Bandstop_Filter_Output[i])

        # ===== Step 5, square the output to improve SNR =====
        # ===== Step 6, detect the R peaks and apply a heuristic to filter incorrect peaks
=====
        detected_R_peaks[i], local_maximum_nmax, was_previous_max =
detect_R_peaks(Matched_FIR_Filter_Output[i] ** 2, min_sample_spacing, was_previous_max,
samples_to_prev_detection, local_maximum_nmax, Fs)

        # If new peak is detected, update heart rate
        if(detected_R_peaks[i]):
            # Only update the BPM once two R_peak readings are done
            if R_detected_once:
                momentary_heart_rate[i] =  60 * Fs / samples_to_prev_detection
            R_detected_once = True
            samples_to_prev_detection = 0
        else:
            if i > 0:
                momentary_heart_rate[i] = momentary_heart_rate[i-1]


    # Plot the Heart Rate vs. Time graph
    plot_time_domain(momentary_heart_rate, time, plot_name + "_Momentary_Heart_Rate",
y_label="Heart Rate (BPM)")

    # ===== Step 7 - Plot the detected R-peaks against the ECG =====

    plt.figure()

    # Plot the original ECG
    plt.plot(time, Input_Bandstop_Filter_Output, label="Bandstop Filtered ECG")

    # Accept only indexes where R is detected
    peak_indexes = np.flatnonzero(detected_R_peaks)

    # Also need to offset the indexes by the R peak of the template itself (just find it's
maximum value)
    R_peak_of_template = np.argmax(template)
    peak_offset = len(template) - 1 - R_peak_of_template
```

```
    peak_indexes -= peak_offset

    # Mark the detected R-peaks with red crosses
    plt.plot(time[peak_indexes], Input_Bandstop_Filter_Output[peak_indexes],'rx',
label="Detected Falling Edge of R-peaks")
    plt.title("Bandstop Filtered ECG with Detected R-peaks")
    plt.xlabel("Time (s)")
    plt.ylabel("Voltage (mV)")
    plt.legend()
    plt.grid()

    # Save as SVG for the report
    plt.savefig(path_to_output + plot_name + "_Original_ECG_With_Detected_R_Peaks.svg")
    plt.show()
    plt.close()

    print("\n===============\n")


if __name__ == "__main__":
    main()
```